Lab 3: Component Modelling & Architectural Pattern Selection

Project: Local Sports Tournament & Bracket Manager

Architecture Selection: We chose Microservices Architecture for the Local Sports Tournament & Bracket Manager System.

Justification

Reason 1 — Independent Scaling:
The system has functionally distinct workloads — team registration, bracket generation, live match scoring, and standings recalculation — that experience very different load patterns during a tournament. Match & Score updates spike heavily during live games, while Team Registration is only active before the tournament starts. Microservices let each of these be deployed and scaled independently, so the Match & Score Service can be scaled up on match day without over-provisioning the rest of the system.

Reason 2 — Fault Isolation:
Tournament operations must continue even if one function fails. Under Microservices, a failure in the Standings & Tiebreaker Service (e.g. during a complex tiebreaker calculation) does not bring down Team Registration or Bracket Management, since each service runs and fails independently. A monolithic or purely layered design would risk a single fault cascading across the entire system during a live event.

Security Advantage:
Each microservice exposes only the specific provided interface it needs (e.g. Team/Seeding API, Score Update API), and communicates with the Database Service through its own required interface. This narrows the attack surface — a vulnerability in one service (such as the public-facing Team Registration Service) does not automatically expose direct database access to the other services, since each service can be given its own scoped credentials and access boundary.

Performance Benefit:
Because services are independently deployable, the Match & Score Service and Standings & Tiebreaker Service can be scaled horizontally on additional instances during peak tournament hours, reducing latency for live score updates, while lower-traffic services such as Team Registration continue running on minimal resources. This targeted scaling improves overall responsiveness compared to scaling an entire monolithic or layered application at once.

Trade-off acknowledged:
Microservices introduce network latency and operational complexity (e.g. keeping standings consistent after each match update), which we manage through clearly defined provided/required interfaces between services, as shown in the accompanying component diagram.